

Earnings Call Intelligence Engine — Complete Project Blueprint

The Big Picture

Publicly traded companies hold quarterly earnings calls where executives discuss financial results, strategy, and outlook. These transcripts are goldmines of unstructured text — and the language executives use (confident vs. hedging, forward-looking vs. defensive) often correlates with future stock performance. You're building a system that turns this unstructured language into structured, actionable intelligence.

What makes this project special

Most NLP portfolio projects stop at sentiment analysis on tweets or movie reviews. This one operates on complex, domain-specific financial language where "soft" signals matter — a CEO saying "we remain cautiously optimistic" vs. "we are confident in our trajectory" carries real meaning. You're showing recruiters that you can handle messy real-world text, build extraction pipelines at scale, and connect NLP outputs to measurable business outcomes.

Project Architecture (3 Levels)

I'd recommend building this in three levels. Level 1 is your MVP — get this done first and it's already a strong project. Levels 2 and 3 add depth if you have time.

Level 1 — Core NLP Pipeline (Week 1)

Goal: Ingest transcripts, extract structured signals, compute sentiment.

Data Collection

SEC EDGAR has free access to earnings call transcripts through 8-K filings. You can also use the Financial Modeling Prep API (free tier gives you 250 calls) or scrape from SeekingAlpha (though be mindful of their terms). Aim for 300–500 transcripts across 10–15 companies over 3–5 years. Pick companies with

interesting stories — ones that had revenue drops, scandals, pivots — because that's where language shifts show up clearly.

Tech stack for Level 1: Python, BeautifulSoup/Scrapy for ingestion, spaCy for NLP, HuggingFace Transformers (FinBERT specifically), Pandas, PostgreSQL.

What you build:

A transcript parser that separates the prepared remarks from the Q&A section — this matters because executives are scripted in remarks but more spontaneous in Q&A. Then a sentiment analysis layer using FinBERT (a BERT model pre-trained on financial text — much better than generic sentiment models for this domain). You compute sentiment at the sentence level, then aggregate by section, speaker, and topic.

Next, build a forward-looking statement detector. The SEC actually defines what forward-looking language looks like — phrases like "we expect," "we anticipate," "our outlook," "going forward." You create a classifier (can start rule-based with spaCy pattern matching, then upgrade to a fine-tuned model) that flags these statements and categorizes their tone: positive outlook, cautious outlook, or hedging.

Key output: For each transcript, you produce a structured JSON with overall sentiment score, sentiment by section (remarks vs. Q&A), count and tone of forward-looking statements, and a list of key extracted entities (revenue figures, guidance numbers, product mentions).

Level 2 — Analytics & Correlation Engine (Week 2)

Goal: Connect language patterns to stock price movement and build temporal analysis.

Tech stack additions: yfinance (for stock data), SciPy/statsmodels (for statistical analysis), Plotly or Streamlit (for visualization).

What you build:

Pull historical stock price data for each company around earnings dates. Compute the stock's reaction in windows: 1-day, 3-day, and 7-day returns after

each earnings call. Then run correlation analysis between your NLP signals and price movement.

The interesting analyses here:

Does Q&A sentiment diverge from prepared remarks sentiment? When it does (executives sound confident in remarks but defensive in Q&A), does that predict negative price movement? This is a genuinely insightful signal that shows analytical thinking.

Build a "language shift detector" — compare consecutive quarterly calls for the same company and flag significant changes in tone, vocabulary, or forward-looking statement frequency. A CEO who goes from "we're thrilled" to "we're working through challenges" is sending a signal. Quantify that shift using cosine similarity between quarter-over-quarter embeddings.

Add topic modeling (BERTopic works well here) to identify what themes dominate each call — are they talking about growth, cost-cutting, competition, regulation? Track how topic distribution shifts over time.

Key output: A correlation report showing which NLP features have predictive value, a temporal dashboard showing sentiment and topic trends per company, and a "divergence alert" system flagging calls where language patterns suggest trouble.

Level 3 — Production-Ready App & Explainability (Week 3)

Goal: Wrap everything in an interactive application with clear explainability.

Tech stack additions: FastAPI (backend), React or Streamlit (frontend), Docker, SHAP or custom attention visualization.

What you build:

An interactive dashboard where a user can select a company, see sentiment trends across quarters, drill into specific calls, and see highlighted passages that drove the sentiment score. The highlighting is important — it's your explainability layer. Don't just say "sentiment = 0.73," show which sentences contributed most and why.

Add a comparison view: pick two companies in the same sector and compare their language patterns. Are they both cautious about the same macro factors? Is one significantly more optimistic?

Build a simple API endpoint where you can submit a new transcript and get back a structured analysis. This shows you're thinking about productionization.

Finally, add a "signal summary" — a one-paragraph auto-generated brief for each earnings call that captures the key takeaways. You can use an LLM for this (tie in your LLaMA experience) or build it with template-based generation from your extracted features.

What to emphasize in the README

Don't start with "this project uses FinBERT to analyze sentiment." Start with the business question: "Can the language patterns in quarterly earnings calls predict short-term stock movement?" Then walk through your findings — what worked, what didn't, what surprised you. Include 2–3 concrete findings with numbers, like "Calls where Q&A sentiment diverged negatively from prepared remarks by >0.3 points preceded negative 3-day returns in 68% of cases." Even if the correlation is weak, discussing *why* it's weak shows analytical maturity.

Resume Bullets (ready to paste)

Pick 2–3 of these:

- "Built an NLP pipeline processing 500+ SEC earnings transcripts using FinBERT and spaCy, extracting sentiment signals, forward-looking statements, and topic distributions across quarterly calls"
 - "Engineered a language shift detection system using sentence embeddings to flag quarter-over-quarter changes in executive tone, correlating divergence patterns with 3-day post-earnings stock returns"
 - "Developed an explainable financial text analytics dashboard with sentence-level sentiment attribution, topic trend visualization, and automated signal summarization via FastAPI and Streamlit"
-

Realistic timeline with your constraints

Days 1–3: Data collection. Get the scraper working, pull 300+ transcripts, parse them into structured format. This is unglamorous but critical — having clean, well-organized data makes everything else faster.

Days 4–7: Level 1 NLP pipeline. Get FinBERT sentiment working, build the forward-looking statement detector, run entity extraction. By end of day 7 you should be able to feed in a transcript and get a structured output.

Days 8–11: Level 2 analytics. Pull stock data, run correlations, build the language shift detector. This is where you'll discover your interesting findings.

Days 12–14: Level 2/3 dashboard and polish. Build the Streamlit app, write the README with your findings, record a short demo. If you have time, Dockerize it.

The key is to have Level 1 fully working by end of week 1. That alone is a presentable project. Everything after that adds depth.

Best Dataset: Kaggle Motley Fool Transcripts

My recommendation: Use the **Scraped Motley Fool Earnings Call Transcripts** dataset from Kaggle.

Why this one wins:

It contains 18,755 earnings call transcripts scraped from Motley Fool [Kaggle](#) — this is the largest freely available dataset by far. Compare that to the alternatives:

- The `jlh-ibm/earnings_call` HuggingFace dataset has only 188 transcripts with 11,970 stock prices, covering 2016–2020 on the NASDAQ [Hugging Face](#). Too small.
- The GeminiLn S&P 500 dataset from ACL19 includes audio recordings alongside text but only covers 2017 and is designed for multimodal research [GitHub](#). Too narrow and complex for your timeline.
- The `lamini/earnings-calls-qa` dataset on HuggingFace contains transcripts with questions and answers in JSONL format [Hugging Face](#), but it's structured for QA tasks, not sentiment analysis.
- The Strux Transcripts dataset has 11,950 transcripts from NASDAQ 500 and S&P 500 companies between 2017 and 2024, with an average of 10,187 tokens per transcript [Struxdata](#). This is a good alternative but smaller.

The Motley Fool dataset gives you the most transcripts, the broadest company coverage, and it's the same source that published research projects use. You'll supplement it with **yfinance** for stock price data (free, no API key needed).

Download link:

<https://www.kaggle.com/datasets/tpotterer/motley-fool-scraped-earnings-call-transcripts>

How to Differentiate from Existing Projects

The existing projects I found all share the same weakness: they treat the transcript as a single blob of text, compute one sentiment score, and correlate it with returns. Here's what makes yours different:

- 1. Section-level analysis (Prepared Remarks vs. Q&A)** The existing project even notes this as a future direction: "Score Q&A sections separately — prepared remarks are IR-optimised; the Q&A is more spontaneous and may carry a stronger signal." [GitHub](#) They didn't do it. You will.
 - 2. Speaker-level analysis** Separate CEO vs. CFO vs. analyst language. Speaker identification can be done using pattern matching against common transcript formatting conventions (e.g. "Tim Cook — CEO" or "Tim Cook (Apple)") [Apify](#).
 - 3. Forward-looking statement detection + language shift tracking** Nobody in the existing projects does this. This is your unique angle.
 - 4. Scale** Existing projects use 56–188 transcripts. You'll use 500+ across 15+ companies.
-

Step-by-Step Execution Plan

Day 1–2: Setup & Data Acquisition

Step 1: Create your project repo and environment.

```
mkdir earnings-call-intelligence && cd earnings-call-intelligence  
python -m venv venv && source venv/bin/activate
```

Step 2: Download the Motley Fool dataset from Kaggle (you'll need a Kaggle account — it's a pickle file ~2GB).

Step 3: Install core dependencies:

```
pip install pandas numpy transformers torch spacy yfinance scipy plotly streamlit scikit-learn  
beautifulsoup4 tqdm  
python -m spacy download en_core_web_sm
```

Step 4: Pull stock price data using yfinance for every ticker in your dataset. Store as a separate CSV.

Day 3–4: Data Preprocessing (THE CRITICAL STEP)

This is where your project lives or dies. Most people rush past this.

What you're building: A parser that takes each raw transcript and produces a structured JSON with:

- `company, ticker, date, quarter`
- `prepared_remarks` → list of `{speaker, role, text}` blocks
- `qa_section` → list of `{speaker, role, text}` blocks (separating analyst questions from executive answers)

How to parse the structure: Motley Fool transcripts follow a consistent format — there's typically a "Prepared Remarks" header and a "Questions and Answers" header. Speaker names appear before their text, usually with a title. You'll use regex patterns to split these sections and identify speakers.

Curate your subset: Don't process all 18K transcripts — pick 15–20 companies across sectors (tech, finance, healthcare, retail, energy), with 5+ years of quarterly calls each. Aim for 400–600 transcripts. Pick companies with interesting stories: companies that had revenue drops, CEO changes, scandals, or major pivots — that's where language shifts show up.

Suggested companies:

- Tech: AAPL, MSFT, META, AMZN, NVDA
- Finance: JPM, GS, BAC
- Healthcare: JNJ, PFE, UNH
- Retail: WMT, TGT, COST
- Energy: XOM, CVX

Day 5–7: Core NLP Pipeline (Level 1)

Component 1: FinBERT Sentiment

FinBERT is a pre-trained NLP model to analyze sentiment of financial text, built by further training BERT using a large financial corpus and fine-tuning it for financial sentiment classification. It gives softmax outputs for three labels: positive, negative or neutral. [Hugging Face](#)

Use `ProsusAI/finbert` from HuggingFace. Run it sentence-by-sentence (not paragraph-level — that's too coarse). Then aggregate by: section (remarks vs. Q&A), speaker, and overall. This gives you a rich feature set, not just one number per transcript.

Also use `yyanghkust/finbert-tone` as a second model — this version is fine-tuned on manually annotated sentences from analyst reports [Hugging Face](#). Having two models and comparing their agreement adds credibility.

Component 2: Forward-Looking Statement (FLS) Detector

Build a rule-based classifier first using spaCy pattern matching. The SEC defines forward-looking language with specific phrases: "we expect," "we anticipate," "our outlook," "going forward," "we believe," "we project," "guidance." Create a list of ~50 such phrases, match them, then classify the tone of the surrounding sentence using FinBERT. Output: count of FLS per transcript, and their average sentiment.

Component 3: Loughran-McDonald Financial Dictionary Scoring

This is the standard benchmark in financial NLP. It provides word lists for: positive, negative, uncertainty, litigious, constraining, superfluous. Score each transcript across all six dimensions. This gives you features beyond just positive/negative.

Day 8–10: Analytics & Correlation (Level 2)

Stock data alignment: Pull price data from yfinance. Compute 1-day, 3-day, and 7-day returns after each earnings call date. Handle timing correctly: if the call was before market open, day-0 is that day; if after close, day-0 is next trading day.

Key analyses to run:

1. **Remarks vs. Q&A divergence:** For each transcript, compute `sentiment_remarks - sentiment_qa`. When executives are more positive in scripted remarks than in spontaneous Q&A, does that predict worse returns? This is your headline finding.
2. **Quarter-over-quarter language shift:** For each company, compute cosine similarity between consecutive quarter embeddings (use sentence-transformers to get transcript-level embeddings). Flag quarters where similarity drops below a threshold — that means the company's narrative changed significantly.
3. **Uncertainty language and returns:** Does increased use of Loughran-McDonald "uncertainty" words predict higher post-earnings volatility? Compute realized volatility over the 5-day window after earnings.
4. **CEO vs. CFO sentiment divergence:** When the CEO is significantly more positive than the CFO, what happens? The CFO deals with numbers and is harder to spin.

Day 11–13: Dashboard & Visualization (Level 3)

Build a **Streamlit** app with these views:

- **Company timeline:** Select a company → see sentiment trend across quarters with stock price overlay. Highlight quarters where sentiment diverged between remarks and Q&A.
- **Transcript deep-dive:** Select a specific call → see sentence-level sentiment heatmap with highlighted FLS. Show which sentences drove the overall score.
- **Correlation dashboard:** Show scatter plots of sentiment features vs. returns, with statistical significance.

- **Language shift alerts:** Show the top 10 most dramatic quarter-over-quarter language shifts in your dataset, with what happened to the stock afterward.

Day 14: Polish, README, Demo

Write the README with your actual findings (numbers, correlations, surprises). Record a 2–3 minute Loom demo walking through the dashboard. Dockerize the app.

Folder Structure

```
earnings-call-intelligence/
├── README.md
├── requirements.txt
├── Dockerfile
├── .gitignore
├── config/
│   └── settings.yaml          # company list, date ranges, model configs
├── data/
│   ├── raw/                  # raw Kaggle pickle file
│   ├── processed/            # parsed structured JSONs
│   ├── market/               # stock price CSVs from yfinance
│   └── features/              # computed NLP feature CSVs
├── src/
│   ├── __init__.py
│   ├── ingestion/
│   │   ├── transcript_parser.py # raw → structured (remarks/QA/speakers)
│   │   └── price_fetcher.py     # yfinance stock data puller
│   ├── nlp/
│   │   ├── finbert_sentiment.py # sentence-level FinBERT scoring
│   │   ├── lm_dictionary.py      # Loughran-McDonald word scoring
│   │   ├── fls_detector.py       # forward-looking statement classifier
│   │   └── embeddings.py         # sentence-transformer embeddings
│   ├── analytics/
│   │   ├── correlation.py        # sentiment vs. returns analysis
│   │   ├── divergence.py        # remarks vs. Q&A, CEO vs. CFO
│   │   └── language_shift.py     # quarter-over-quarter tracking
│   └── app/
│       └── streamlit_app.py      # dashboard
└── notebooks/
    ├── 01_data_exploration.ipynb
    ├── 02_preprocessing.ipynb
    ├── 03_sentiment_analysis.ipynb
    └── 04_correlation_analysis.ipynb
```

```
|   └─ 05_findings_summary.ipynb
|   └─ tests/
|       └─ test_parser.py
|       └─ test_sentiment.py
```

Tech Stack Summary

Layer	Tools
Data	Kaggle Motley Fool dataset, yfinance, Pandas
NLP	ProsusAI/FinBERT, yiyanghkust/finbert-tone, spaCy, Loughran-McDonald dictionary, sentence-transformers
Analytics	SciPy (Spearman correlation), statsmodels, scikit-learn
Visualization	Streamlit, Plotly
Infrastructure	Docker, Git

What to do right now

1. Create a Kaggle account if you don't have one
2. Download the dataset: `kaggle datasets download tpotterer/motley-fool-scraped-earnings-call-transcripts`
3. Set up your repo with the folder structure above
4. Start with `transcript_parser.py` — load the pickle file, explore the format, and figure out the regex patterns for splitting sections and identifying speakers